Day 125 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 7 reference

Watch: StatQuest: t-SNE, Clearly Explained (StatQuest with Josh Starmer) | UMAP explained | The best dimensionality reduction? (AI Coffee Break with Letitia)

`day125_tsne_umap.py`, segment by segment, in the order `main()` uses things. The six exercise parts (Days 119-124) are copied verbatim and were explained on their own days; this guide recaps them in one segment, then walks the new code: t-SNE, a compact UMAP, the measures that judge a map, and the experiments. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: P = neighbour probabilities in the original space · Q = similarities on the page · Y = the 2-D layout · W = UMAP's weighted graph · a, b = UMAP's curve · KL = Kullback-Leibler divergence

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
NOISE = -1
LOG2PI = np.log(2 * np.pi)
PERPLEXITY = 30
NEIGHBOURS = 15
MIN_DIST = 0.1
```

- `NOISE = -1; LOG2PI = ...` — two constants the copied parts of Days 121 and 122 use, defined here as on their own days.
- `import umap` — the umap-learn library, used only to check the compact UMAP written today.
- `PERPLEXITY = 30` — t-SNE's effective number of neighbours per point.

Segment 1 — The week's parts

The week's parts, verbatim (one banner per exercise file)

CODE

```
# from day119_exercise.py, verbatim: sqdist, init_centres, lloyd, kmeans, silhouette, ari, elbow
# ============================================================================
def sqdist(A, B):                                       # every squared distance
...  (Days 120 to 124 follow, each under its own banner)
```

- `# from dayNNN_exercise.py, verbatim: ...` — a generator pasted the 26 parts in; Days 120 and 121's copies of sqdist were left out as the same code.
- `kmeans, agglomerate, dbscan, e_step, pca_fit, ...` — today PCA starts t-SNE, and four of the parts cluster the finished maps.

Segment 2 — t-SNE

p_conditional and joint_p: the neighbour probabilities

MATH

$$p_{j|i} = exp(-\beta_i |x_i - x_j|^2) / \Sigma_{k \neq i} exp(-\beta_i |x_i - x_k|^2); \quad perplexity = e^H, H = -\Sigma\, p\, log\, p$$

$$p_{ij} = (p_{j|i} + p_{i|j}) / 2n$$

CODE

```
def p_conditional(D2, perplexity, iters=64, tol=1e-6):
    n = len(D2); beta = np.ones(n); lo = np.zeros(n); hi = np.full(n, np.inf); target =
        np.log(perplexity)
    off = ~np.eye(n, dtype=bool)
    E = np.where(off, D2 - np.where(off, D2, np.inf).min(1, keepdims=True), 0.0)
    for _ in range(iters):
        P = np.where(off, np.exp(-E * beta[:, None]), 0.0); s = P.sum(1)
        H = np.log(s) + beta * (P * E).sum(1) / s
        too_flat = H > target
        lo = np.where(too_flat, beta, lo); hi = np.where(too_flat, hi, beta)
        beta = np.where(np.isinf(hi), beta * 2, (lo + hi) / 2)
        if np.abs(H - target).max() < tol:
            break
    return P / s[:, None], np.exp(H)
def joint_p(X, perplexity):
    D2 = np.maximum(sqdist(X, X), 0); Pc, perp = p_conditional(D2, perplexity)
    return np.maximum((Pc + Pc.T) / (2 * len(X)), 1e-12), perp
```

- `off = ~np.eye(n, dtype=bool)` — True everywhere except the diagonal: a point is never its own neighbour.
- `lo, hi, beta: np.where(too_flat, ...)` — a binary search run on every row at once; hi starts at infinity, so beta doubles until a bracket is found.
- `np.log(s) + beta * (P * E).sum(1) / s` — each row's entropy; its exponential is the perplexity, the effective number of neighbours.
- `(Pc + Pc.T) / (2 * len(X))` — symmetric: $p_{ij} = p_{ji}$, and all entries sum to 1.

tsne_grad and tsne

MATH

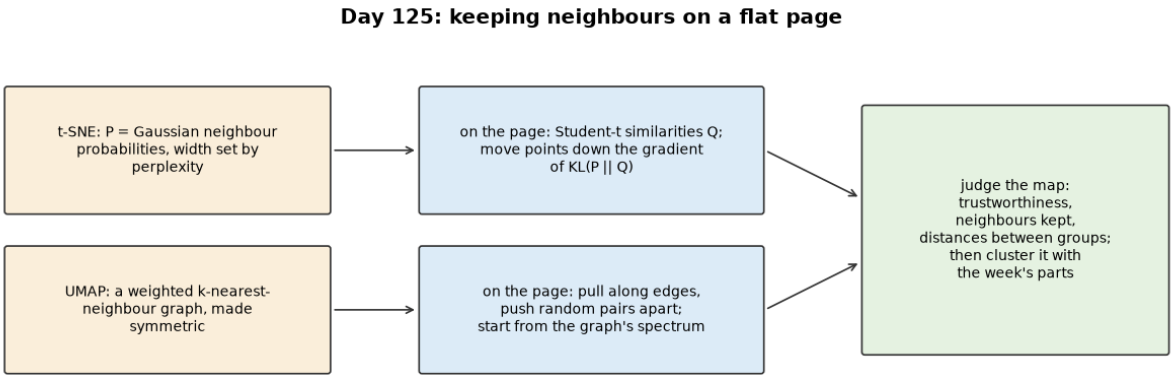
$$q_{ij} = (1 + |y_i - y_j|^2)^{-1}/Z; \quad KL = \Sigma\, p_{ij}\, log(p_{ij}/q_{ij}); \quad \partial KL/\partial y_i = 4\Sigma_j(p_{ij} - q_{ij})(y_i - y_j)(1 + |y_i - y_j|^2)^{-1}$$

CODE

```
def tsne_grad(Y, P):
    num = 1.0 / (1.0 + np.maximum(sqdist(Y, Y), 0)); np.fill_diagonal(num, 0)
    Q = np.maximum(num / num.sum(), 1e-12); PQ = (P - Q) * num
    grad = 4 * (PQ.sum(1)[:, None] * Y - PQ @ Y)
    return float((P * np.log(P / Q)).sum()), grad
def tsne(X, perplexity=PERPLEXITY, iters=1000, seed=0, exaggeration=12.0, exag_iters=250):
    n = len(X); P, _ = joint_p(X, perplexity); lr = max(n / exaggeration / 4, 50)
    mu, vals, V = pca_fit(X); Y = transform(X, mu, V, 2); Y = Y / Y[:, 0].std() * 1e-4
    upd = np.zeros_like(Y); gains = np.ones_like(Y); kl = []
    for t in range(iters):
        ex = exaggeration if t < exag_iters else 1.0
        _, g = tsne_grad(Y, P * ex)
        gains = np.where(upd * g < 0, gains + 0.2, gains * 0.8).clip(0.01)
        upd = (0.5 if t < exag_iters else 0.8) * upd - lr * gains * g
        Y = Y + upd
        if (t + 1) % 50 == 0:
            kl.append(tsne_grad(Y, P)[0])
    return Y, kl
```

- `num = 1.0 / (1.0 + ...)` — Student-t similarities on the page: heavy tails let distant groups sit far apart.
- `4 * (PQ.sum(1)[:, None] * Y - PQ @ Y)` — the gradient for every point at once: the sum over j written as two matrix products.
- `np.where(upd * g < 0, gains + 0.2, gains * 0.8)` — per-coordinate step sizes: grow while the direction holds, shrink when it flips.
- `ex = exaggeration if t < exag_iters else 1.0` — early exaggeration: P multiplied by 12 for 250 steps, so groups form before details.

SYSTEM DIAGRAM



t-SNE and UMAP: neighbours in, neighbours out.

Segment 3 — UMAP

find_ab and fuzzy_graph

MATH

$w_{ij} = exp(-(d_{ij} - \rho_i)/\sigma_i)$, $\Sigma_j\, w_{ij} = log_2\, k$; *on the page:* $1/(1 + a\, d^{2b})$

CODE

```
def find_ab(min_dist=MIN_DIST, spread=1.0):
    d = np.linspace(0, 3 * spread, 300); f = np.where(d < min_dist, 1.0, np.exp(-(d - min_dist) /
        spread))
    best = (np.inf, None)
    for b in np.linspace(0.5, 1.5, 201):
        for a in np.exp(np.linspace(np.log(0.5), np.log(5.0), 201)):
            e = ((1 / (1 + a * d ** (2 * b)) - f) ** 2).sum()
            if e < best[0]:
                best = (e, (a, b))
    return best[1]
def fuzzy_graph(X, k=NEIGHBOURS):
    n = len(X); D = np.sqrt(np.maximum(sqdist(X, X), 0)); np.fill_diagonal(D, np.inf)
    idx = np.argsort(D, 1)[:, :k]; d = np.take_along_axis(D, idx, 1)
    rho = d[:, 0]; lo = np.zeros(n); hi = np.full(n, np.inf); sig = np.ones(n); target =
        np.log2(k)
    for _ in range(64):
        s = np.exp(-np.maximum(d - rho[:, None], 0) / sig[:, None]).sum(1)
        big = s > target
        hi = np.where(big, sig, hi); lo = np.where(big, lo, sig); sig = np.where(np.isinf(hi), sig
            * 2, (lo + hi) / 2)
    W = np.zeros((n, n)); W[np.arange(n)[:, None], idx] = np.exp(-np.maximum(d - rho[:, None], 0)
        / sig[:, None])
    return W + W.T - W * W.T
```

- `np.take_along_axis(D, idx, 1)` — picks, row by row, the distances at the positions in idx.
- `W[np.arange(n)[:, None], idx] = ...` — fancy indexing with a column of row numbers: fills each row's k chosen columns at once.
- `W + W.T - W * W.T` — the fuzzy union: if either point sees the other, the edge exists; both, and it is stronger.

spectral_init and umap_layout

CODE

```
def spectral_init(W, dims=2):
    deg = W.sum(1); Dm = 1 / np.sqrt(deg); L = np.eye(len(W)) - Dm[:, None] * W * Dm[None, :]
    vals, vecs = np.linalg.eigh(L); Y = vecs[:, 1:dims + 1]
    return 10 * Y / np.abs(Y).max()
def umap_layout(W, a, b, epochs=500, neg=5, seed=0):
    rng = np.random.RandomState(seed); Y = spectral_init(W); n = len(W)
    i, j = np.nonzero(np.triu(W)); w = W[i, j] / W[i, j].max()
    for e in range(epochs):
        alpha = 1.0 - e / epochs
        use = rng.uniform(size=len(w)) < w
        a_i, a_j = i[use], j[use]; diff = Y[a_i] - Y[a_j]; d2 = (diff ** 2).sum(1)
        c = (-2 * a * b * d2 ** (b - 1)) / (1 + a * d2 ** b); c[d2 == 0] = 0
        g = np.clip(c[:, None] * diff, -4, 4)
        dY = np.zeros_like(Y); np.add.at(dY, a_i, alpha * g); np.add.at(dY, a_j, -alpha * g)
        r_i = np.repeat(a_i, neg); r_k = rng.randint(n, size=len(r_i))
        diff = Y[r_i] - Y[r_k]; d2 = (diff ** 2).sum(1)
        c = 2 * b / ((0.001 + d2) * (1 + a * d2 ** b)); c[r_i == r_k] = 0
        np.add.at(dY, r_i, alpha * np.clip(c[:, None] * diff, -4, 4))
        Y = Y + dY
    return Y
def umap_embed(X, k=NEIGHBOURS, min_dist=MIN_DIST, epochs=500, seed=0):
    a, b = find_ab(min_dist); return umap_layout(fuzzy_graph(X, k), a, b, epochs, seed=seed)
```

- `np.eye(len(W)) - Dm[:, None] * W * Dm[None, :]` — the normalised graph Laplacian; its smallest eigenvectors are the graph's smoothest directions.
- `use = rng.uniform(size=len(w)) < w` — each edge takes part in an epoch with probability equal to its weight.
- `np.add.at(dY, a_i, alpha * g)` — adds every edge's push to its endpoint, counting repeated endpoints.
- `np.repeat(a_i, neg)` — each used edge's first point, repeated for its random 'negative' partners.

Segment 4 — Measures and experiments

Judging a map: trustworthiness, kept and centre_corr

MATH

$$T(k) = 1 - 2/(nk(2n - 3k - 1)) \, \Sigma_i \, \Sigma_{j \in N_k(i) \text{ on the map}} \, max(0, r(i, j) - k)$$

CODE

```
def knn_sets(X, k):
    D = np.maximum(sqdist(X, X), 0); np.fill_diagonal(D, np.inf); return np.argsort(D, 1)[:, :k],
        D
def trust(X, Y, k=10):
    n = len(X); _, DX = knn_sets(X, k); ranks = np.argsort(np.argsort(DX, 1), 1) + 1
    NY, _ = knn_sets(Y, k); r = np.take_along_axis(ranks, NY, 1)
    return float(1 - 2 / (n * k * (2 * n - 3 * k - 1)) * np.maximum(r - k, 0).sum())
def kept(X, Y, k=10):
    NX, _ = knn_sets(X, k); NY, _ = knn_sets(Y, k)
    return float(np.mean([len(set(a) & set(b)) / k for a, b in zip(NX, NY)]))
def centre_corr(X, Y, y):
    cx = np.array([X[y == c].mean(0) for c in range(10)]); cy = np.array([Y[y == c].mean(0) for c
        in range(10)])
    iu = np.triu_indices(10, 1); return float(np.corrcoef(np.sqrt(sqdist(cx, cx)[iu]),
        np.sqrt(np.maximum(sqdist(cy, cy), 0)[iu]))[0, 1])
```

- `np.argsort(np.argsort(DX, 1), 1) + 1` — ranks: for every pair, how near j is to i in the original space (1 = nearest).

- `np.maximum(r - k, 0).sum()` — a map neighbour that was not among the k nearest originally costs how far down the list it was.
- `len(set(a) & set(b)) / k` — set intersection: the share of a point's original neighbours still beside it.

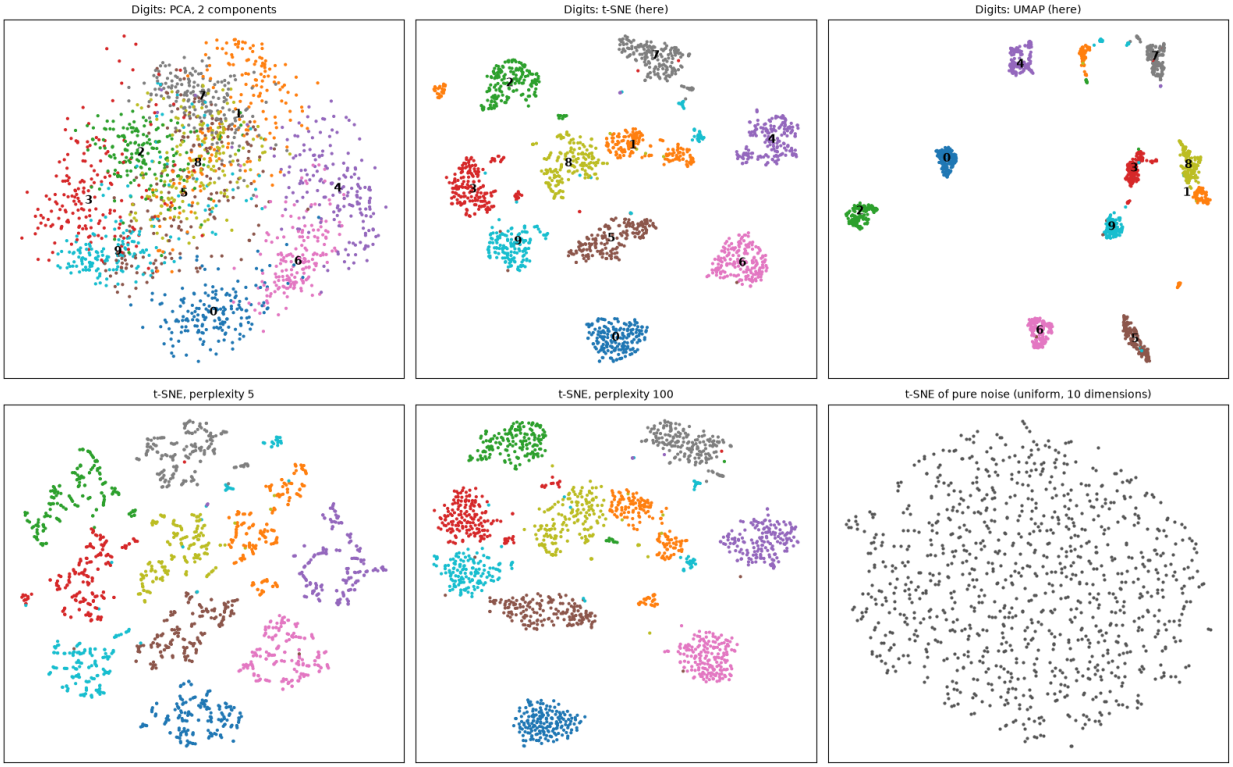
cluster_all: the week's four clusterers

CODE

```
def gmm(X, k, seed=0, reg=1e-6, iters=300):
    R = np.eye(k)[kmeans(X, k, seed)[0]]; prev = -np.inf
    for _ in range(iters):
        w, M, S = m_step(X, R, "full", reg); R, ll = e_step(X, w, M, S)
        if ll - prev < 1e-6 * len(X):
            break
        prev = ll
    return R.argmax(1)
def cluster_all(Z, y):
    eps = float(np.percentile(k_distance(Z, 10), 90))
    out = {"k-means": kmeans(Z, 10, 0)[0], "Ward": cut(agglomerate(Z, "ward"), len(Z), 10),
        "Gaussian mixture": gmm(Z, 10),
           "DBSCAN": dbscan(Z, eps, 10)[0]}
    return {m: ari(lab, y) for m, lab in out.items()}, out["DBSCAN"], eps
```

- `np.eye(k)[kmeans(X, k, seed)[0]]` — Day 119's k-means labels as hard responsibilities: Day 122's start.
- `np.percentile(k_distance(Z, 10), 90)` — DBSCAN's eps from Day 121's k-distance: the 90th percentile, a rule fixed before seeing the results.
- `{m: ari(lab, y) for m, lab in out.items()}` — a dictionary comprehension: every method's ARI at once.

REAL GRAPH



Real graph from the script: three maps of the digits, two perplexities, and t-SNE of pure noise.

Quick Syntax Glossary — new or reinforced constructs

- `np.take_along_axis(D, idx, 1)` — pick each row's entries at given positions.
- `A[rows[:, None], cols]` — fill each row's chosen columns at once.
- `np.add.at(a, idx, v)` — add with repeated indices counted.
- `np.repeat(a, k)` — each element k times in a row.
- `set(a) & set(b)` — the elements in both.
- `np.percentile(a, 90)` — the value below which 90% fall.

End of Day 125 syntax reference. For the experiments and what they showed, see the Day 125 complete guide.